

Florence: Infrastructure - GitHub Repository

1. Overview and Purpose

- Repository Name:** Swinburne-COS40005-CTPA-FLORENCE
- Repository URL:** https://github.com/darklorddad/Swinburne-COS40005-CTPA-FLORENCE
- Purpose:** This repository serves as the single source of truth and centralised version control system for the entire Florence platform. It is structured as a monorepo containing the cross-platform Flutter frontend (Patient, Clinician and Admin dashboards) and the Python-based backend microservices.

2. Repository Structure (The Monorepo)

The codebase is organised into distinct service folders to separate concerns whilst keeping all project artefacts centralised:

- florence/platform_service/** : The Flutter frontend application. Contains the Patient, Clinician and Admin dashboards. Includes shorebird.yaml for mobile OTA updates and vercel.json for web hosting.
- florence/data_service/** : Python FastAPI microservice. Handles all CRUD operations, Supabase database proxying, role-based access control (RBAC) and file uploads.
- florence/llm_chatbot_service/** : Python FastAPI microservice. Powers the patient-facing conversational AI chatbot using LangChain.
- florence/llm_engine_service/** : Python FastAPI microservice. Handles background AI tasks including nutrition analysis (meal photos), risk assessment, biometric parsing and daily insight generation.
- Project-documentation/** : Contains the compiled User Manuals, Architecture diagrams and Handover documents.
- .github/** : Contains GitHub Actions workflows (.yaml files) that automate CI/CD pipelines.

3. Branching Strategy and Version Control

The team utilises a streamlined branching model optimised for continuous integration, rapid deployment and repository hygiene:

- main branch:** The primary integration branch for all ongoing development. All new features, bug fixes and updates are merged here. This represents the latest stable development state.
- production branch:** The stable, release-ready branch. Code is strictly merged from main to production when a milestone is reached or a stable release is ready for deployment to live production environments.
- Feature and Backup Branches:** Created dynamically from main for isolated feature development or as backup snapshots before major refactors. To maintain a clean repository, these branches are constantly managed and deleted once they are merged or no longer needed.
- Pull Requests (PRs):** Used to merge feature branches into main to allow for code review.

4. GitHub Environments and Vercel Integration

To manage deployments securely and separate staging from live systems, the repository utilises GitHub Environments. These environments are directly managed and triggered by Vercel to handle continuous deployment for the backend microservices and the web frontend.

Environment Name	Purpose	Target Service
Preview - ds	Staging and preview deployment for the Data Service	data_service
Production - ds	Live production deployment for the Data Service	data_service
Preview - llmcs	Staging and preview deployment for the Chatbot Service	llm_chatbot_service
Production - llmcs	Live production deployment for the Chatbot Service	llm_chatbot_service
Preview - llmes	Staging and preview deployment for the AI Engine	llm_engine_service

Environment Name	Purpose	Target Service
Production - llmes	Live production deployment for the AI Engine	llm_engine_service
Production - web	Live production deployment for Flutter Web	platform_service

Vercel monitors the repository and automatically routes deployments to the correct GitHub Environment based on the branch and directory structure. This ensures that preview links are generated for pull requests whilst production environments are strictly updated only when code is merged into the designated release branches.

5. Secrets Management

GitHub Secrets are used to securely pass credentials to GitHub Actions without exposing them in the source code. The following **Repository-level Secrets** are currently configured:

Secret Name	Description and Usage
CREDENTIAL_FILE_CONTENT	Used for authenticating backend services or CI/CD pipelines with cloud providers.
FIREBASE_APP_ID	The unique identifier for the Firebase project, used by CI/CD to target the correct project for App Distribution (beta testing).
FIREBASE_SERVICE_ACCOUNT_JSON_BASE64	Base64 encoded service account key. Allows GitHub Actions to authenticate with Firebase securely to deploy builds or manage App Distribution.
FIREBASE_TOKEN	CLI token used for Firebase hosting deployments or automated testing environments.
SHOREBIRD_TOKEN	Authentication token for the Shorebird CLI. Allows GitHub Actions to push Over-The-Air (OTA) patches to the Flutter mobile app.

6. Local Development Setup (Quick Start)

The following steps outline the process for a new developer to clone and work with the repository:

1. **Repository Cloning:**

```
git clone https://github.com/darklordddad/Swinburne-COS40005-CTPA-FLORENCE.git
cd Swinburne-COS40005-CTPA-FLORENCE
```

2. **Branch Checkout:**

```
git checkout main
```

3. **Fetching Latest Changes:**

```
git pull origin main
```

The respective backend and frontend documentation should be referenced for environment setup, dependency installation and local execution commands.